AI-Powered Demand Forecasting & Knowledge Assistant

Problem Statement

To develop an intelligent AI system that predicts demand for electronic products and answers user queries using machine learning, retrieval-based search (RAG), and large language models.

Businesses often struggle to accurately predict product demand and analyze large datasets manually. This project aims to develop an Agentic AI system that automates demand prediction for electronic devices using features like price, rating, brand, and location, while also enabling intelligent query answering by retrieving relevant data through embeddings and generating responses using a large language model.

Objectives

- Predict product demand using ML model
- Process and clean real-world dataset
- Convert text data into embeddings
- Build FAISS-based search system
- Generate answers using LLM (Gemini)
- Implement decision agent for smart routing
- Create chatbot interface for user interaction

1. Imports

```
# Core
import pandas as pd
import numpy as np

# ML
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
```

```
# Embeddings + Vector DB
from sentence_transformers import SentenceTransformer
!pip install faiss-cpu
!pip install sentence-transformers
import faiss

# Gemini (ONLY THIS)
from google import genai
from google.colab import userdata
```

```
Requirement already satisfied: transformers<6.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (5.0.0)
Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.10.1)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.10.0+cu128)
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.0.2)
Requirement already satisfied: scikit-learn>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.16.3)
Requirement already satisfied: typing_extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.15.0)
Requirement already satisfied: tqdm>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.67.3)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transforme
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (6.0
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (0.24.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence-transformers) (1.5.3
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence-transformers
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.1.6)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (12.9
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transforr
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transfo
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transforr
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformer
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transforr
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transforr
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformer
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (2
```

```
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch>=1.11.0->senter
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence-transforr
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence-transformers) (0
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence-transfor
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transfo
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-trans
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transfor
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.23
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.11.0->sentence-transfo
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.11.0->sentence-transformers) (3
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.23.0->sentence-transformer
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.23.0->sentence-trans
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.23.0->sentence-transformer
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.23.0->sentence-tra
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub>=0.23
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub>=0.2
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingfac
```

- **Core Libraries**

pandas (pd) → handle dataset (table format)

numpy (np) → perform numerical calculations

- **Machine Learning**

train_test_split → divide data (train + test)

StandardScaler → scale data (same range)

RandomForestRegressor → predict demand using multiple trees

mean_absolute_error (MAE) → check prediction accuracy

- **Embeddings + FAISS**

SentenceTransformer → convert text → numerical vectors

faiss → fast similarity search (find related data)

- **Installation**

pip install → installs required libraries

- **Gemini AI**

genai → connect to Gemini model (AI responses)

userdata → safely store API key

2. Gemini Setup

We fetch API key securely and initialize Gemini client for AI communication. 👍

```
API_KEY = userdata.get('GOOGLE_API_KEY')
client = genai.Client(api_key=API_KEY)
```

3. Data Preprocessing

Load dataset, clean missing & duplicate data, encode categorical features, split into input/output, and scale features for ML model training. 👍

```
df = pd.read_csv("/content/drive/MyDrive/Agentic AI/electronic_device_data.csv")

# Drop unwanted
df.drop(columns=['Source_File', 'Source_Demand_Level'], errors='ignore', inplace=True)

# Fill missing
numeric_cols = df.select_dtypes(include=np.number).columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())

categorical_cols = df.select_dtypes(include='object').columns
for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

# Remove duplicates
df.drop_duplicates(inplace=True)

# One-hot encoding (ONLY THIS)
df = pd.get_dummies(df, drop_first=True)

# Split
target_column = 'Demand_Score'
```

```
X = df.drop(columns=[target_column])
y = df[target_column]

# Scale
scaler = StandardScaler()
X = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
```

4. Train Model

Split data into training and testing, train Random Forest model, make predictions, and evaluate accuracy using MAE. 👍

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

preds = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, preds))
```

```
MAE: 0.0199451242409121
```

5. Prediction Function

Convert user input to model format, match all features, scale it, and use trained model to predict demand. 👍

```
def predict_demand(input_dict):
    input_df = pd.DataFrame([input_dict])

    # Ensure all columns exist
    for col in X.columns:
        if col not in input_df:
            input_df[col] = 0

    input_df = input_df[X.columns]
```

```
    scaled = scaler.transform(input_df)
    return model.predict(scaled)[0]
```

6. Embedding + FAISS

Create text embeddings from dataset and store them in FAISS index for fast similarity-based retrieval. 👍

```
from google.colab import userdata
import os
from huggingface_hub import login

os.environ["HF_TOKEN"] = userdata.get('HF_TOKEN')
embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

df['text'] = df.astype(str).agg(" ".join, axis=1)
documents = df['text'].tolist()

embeddings = embedding_model.encode(documents)
embeddings = np.array(embeddings).astype('float32')

index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)
```

```
Loading weights: 100%|| 103/103 [00:00<00:00, 518.85it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

7. Retrieval

Convert user query into embedding, search FAISS index, and return top similar documents. 👍

```
def retrieve_docs(query, top_k=10):
    query_vec = embedding_model.encode([query])
    D, I = index.search(np.array(query_vec).astype('float32'), top_k)
    return [documents[i] for i in I[0]]
```

8. LLM Response

```
def llm_response(prompt):
    response = client.models.generate_content(
        model="gemini-flash-latest", # Changed to gemini-flash-latest
        contents=prompt
    )
    return response.text
```

```
for m in client.models.list():
    print(m.name)
```

```
models/gemini-2.5-flash
models/gemini-2.5-pro
models/gemini-2.0-flash
models/gemini-2.0-flash-001
models/gemini-2.0-flash-lite-001
models/gemini-2.0-flash-lite
models/gemini-2.5-flash-preview-tts
models/gemini-2.5-pro-preview-tts
models/gemma-3-1b-it
models/gemma-3-4b-it
models/gemma-3-12b-it
models/gemma-3-27b-it
models/gemma-3n-e4b-it
models/gemma-3n-e2b-it
models/gemma-4-26b-a4b-it
models/gemma-4-31b-it
models/gemini-flash-latest
models/gemini-flash-lite-latest
models/gemini-pro-latest
models/gemini-2.5-flash-lite
models/gemini-2.5-flash-image
models/gemini-3-pro-preview
models/gemini-3-flash-preview
models/gemini-3.1-pro-preview
models/gemini-3.1-pro-preview-customtools
models/gemini-3.1-flash-lite-preview
```

```
models/gemini-3-pro-image-preview
models/nano-banana-pro-preview
models/gemini-3.1-flash-image-preview
models/lyria-3-clip-preview
models/lyria-3-pro-preview
models/gemini-robotics-er-1.5-preview
models/gemini-robotics-er-1.6-preview
models/gemini-2.5-computer-use-preview-10-2025
models/deep-research-pro-preview-12-2025
models/gemini-embedding-001
models/gemini-embedding-2-preview
models/aqa
models/imagen-4.0-generate-001
models/imagen-4.0-ultra-generate-001
models/imagen-4.0-fast-generate-001
models/veo-2.0-generate-001
models/veo-3.0-generate-001
models/veo-3.0-fast-generate-001
models/veo-3.1-generate-preview
models/veo-3.1-fast-generate-preview
models/veo-3.1-lite-generate-preview
models/gemini-2.5-flash-native-audio-latest
models/gemini-2.5-flash-native-audio-preview-09-2025
models/gemini-2.5-flash-native-audio-preview-12-2025
models/gemini-3.1-flash-live-preview
```

9. RAG Agent

```
def rag_agent(query):
    docs = retrieve_docs(query)
    context = "\n".join(docs)

    prompt = f"""
Use this data:

{context}

Answer:
{query}
"""

    return llm_response(prompt)
```

10. NLP → Prediction Agent

Use LLM to extract structured features from query in JSON format and return them for prediction. 👍

```
import json

def extract_features(query):
    prompt = f"""
Extract features as JSON.

Query: {query}
"""
    res = llm_response(prompt)

    try:
        return json.loads(res)
    except:
        return {}
```

Extract features from query, convert to model input format, and predict demand using the trained model. 👍

```
def prediction_agent(query):
    data = extract_features(query)

    input_dict = {col: 0 for col in X.columns}

    for k, v in data.items():
        if k in input_dict:
            input_dict[k] = v
        else:
            col_name = f"{k}_{v}"
            if col_name in input_dict:
                input_dict[col_name] = 1

    demand = predict_demand(input_dict)

    return f"Predicted Demand: {demand}"
```

11. Decision Agent

Check user query and decide whether to use prediction model or RAG-based answer system. 👍

```
def decision_agent(query):
    if "predict" in query.lower() or "forecast" in query.lower():
        return prediction_agent(query)
    else:
        return rag_agent(query)
```

12. Chatbot

Continuously take user input, process it using decision agent, and display the chatbot response until exit. 👍

```
def chatbot():
    while True:
        q = input("You: ")
        if q == "exit":
            break
        print("Bot:", decision_agent(q))

chatbot()
```

```
/ou: What is the demand trend for mobile phones?
3ot: Based on the data provided, the demand trend for mobile phones shows a **significant overall decrease** between 2021 and 2024.

### Key Observations:
*   **Peak Demand (2021):** The highest demand occurred in early 2021, with a peak volume of **201,433 units** in January and another high of **16
*   **Declining Volume:** By 2023, the sample shows a volume of **73,039 units** (April). By May 2024, the demand recorded in the data dropped to
*   **Revenue Correlation:** Total revenue (Column 6) followed a similar downward trajectory, falling from a high of **54,039.65** in early 2021 t
*   **Stability in Ratings:** Despite the drop in demand, consumer satisfaction remained relatively stable, with ratings (Column 7) consistently r

### Summary of Demand by Year (Samples Provided):
*   **2021:** High volatility but very high peaks (reaching >200k).
*   **2022:** Moderate demand (approx. 20k in the August sample).
*   **2023:** Moderate demand (approx. 73k in the April sample).
*   **2024:** Low demand (approx. 8.5k in the May sample).

**Conclusion:** The data indicates a **contracting market** or a sharp decline in sales volume for the specific category/models tracked over this
/ou: Which brand is most popular?
```

```
Bot: Based on the dataset provided, which appears to be a one-hot encoded dataset for car attributes (commonly used in "Used Car Price Prediction"

In this specific dataset format:
1.  **Honda** is represented by the **2nd** boolean column (Index 1).
2.  **Toyota** is represented by the **5th** boolean column (Index 4).
3.  **BMW** is represented by the **1st** boolean column (Index 0).

**Frequency count from the 10 rows provided:**
*   **Honda (Index 1):** Appears in Rows 4, 6, 8, and 10 (**4 times**).
*   **Toyota (Index 4):** Appears in Rows 1, 2, and 3 (**3 times**).
*   **BMW (Index 0):** Appears in Row 9 (**1 time**).
*   **Other:** Rows 5 and 7 represent a different brand (Index 24).

The most popular brand in this data is **Honda**.
You: Predict demand for Samsung TV in Mumbai
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestRegres
  warnings.warn(
Bot: Predicted Demand: 0.6158729629344614
You: What are low-demand products?
Bot: Based on the data provided, we can identify **low-demand products** by looking at the **fourth column**, which represents the quantity sold (

In this dataset, the demand values are: 14, 15, **3**, 20, 22, 15, **1**, 14, 16, and **5**.

The low-demand products (those with the lowest quantity sold) are:

1.  **Product ID 64409 (Row 7):** Only **1** unit sold.
    *   *Context:* Sold in Month 12, despite having a high rating of 4.8.
2.  **Product ID 8631 (Row 3):** Only **3** units sold.
    *   *Context:* Sold in Month 11 with a rating of 4.2.
3.  **Product ID 196215 (Row 10):** Only **5** units sold.
    *   *Context:* Sold in Month 11 with a rating of 4.0.

### Comparison Summary:
*   **High Demand:** Product ID 70820 (22 units) and Product ID 191043 (20 units).
*   **Low Demand:** Product IDs 64409, 8631, and 196215 (between 1 and 5 units).

**Note on Revenue:**
Low demand does not always mean low revenue. For example, Product ID 196215 sold only 5 units but generated \$54,186.40, suggesting it is a high-v
You: How does price affect demand?
Bot: Based on the data provided, the relationship between **Price** (Column 5) and **Demand** (Column 6) is heavily influenced by the **Market Vol
```

⭐ Simple What products have highest demand?

⭐ Medium Which city has highest demand for laptops?

⭐ Advanced Predict demand for laptop with price 60000 and rating 4.5

⭐ PRO LEVEL 🔥 Based on data, suggest how to increase demand for smartphones

🔹 1. 📊 Data Insight Questions (RAG based)

These use your FAISS + embeddings + Gemini

Examples: What products have highest demand?

Which city has highest demand for laptops?

What is the demand trend for mobile phones?

Which brand is most popular?

How does price affect demand?

What are low-demand products?

👉 These questions = data analysis + business insights

🔹 2. 🤖 Prediction Questions (ML based)

These use your Random Forest model

Examples: Predict demand for laptop with price 60000 and rating 4.2

Forecast demand for mobile with price 20000

What will be demand if discount is 30%?

Predict demand for Samsung TV in Mumbai

How will rating impact demand?

👉 These = real-world business decision questions

🔹 3. 🧠 Intelligent Questions (LLM + Agent)

These show your Agentic AI power

Examples: Should I increase price or discount to improve demand?

Which product should I stock more? What business strategy should I use for high demand?

Explain demand trend in simple words Give recommendations for increasing sales

👉 These = HIGH LEVEL AI thinking (very impressive)

```
Start coding or generate with AI.
```